

LOGEART Pierre, ADAM Tristan, MILOIKOVITCH Maxime, MARCOLÉ Mattéo – RA-IL2

lien du git :

https://github.com/plogeart/RMI_Calcul_Paral-1le.git

Préambule

Question

En utilisant votre meilleur outil, votre imagination, décrivez et illustrez comment cela pourrait être réalisé, sans rentrer dans les détails JAVA, que vous n'allez pas tarder à mettre en œuvre.

Le calcul parallèle consiste à répartir un gros calcul en plusieurs petits calculs indépendants, qui peuvent être exécutés en même temps par plusieurs machines. L'objectif est de réduire le temps total d'exécution en utilisant plusieurs processeurs au lieu d'un seul.

Dans le cas d'un calcul très lourd, dont la synthèse d'une image par raytracing pour ce projet, une seule machine doit normalement calculer tous les pixels de l'image. Plus l'image est grande, plus le nombre de pixels augmente, et donc plus le temps de calcul devient important. L'idée est donc de découper l'image en plusieurs morceaux, puis de confier chaque morceau à une machine différente. Chaque machine calcule uniquement sa partie, puis renvoie le résultat. Un programme principal récupère ensuite toutes les parties calculées et les assemble pour former l'image finale.

Chaque nœud reçoit alors :

- la scène à calculer
- la taille de l'image
- la zone dont il est responsable
- les paramètres de calcul

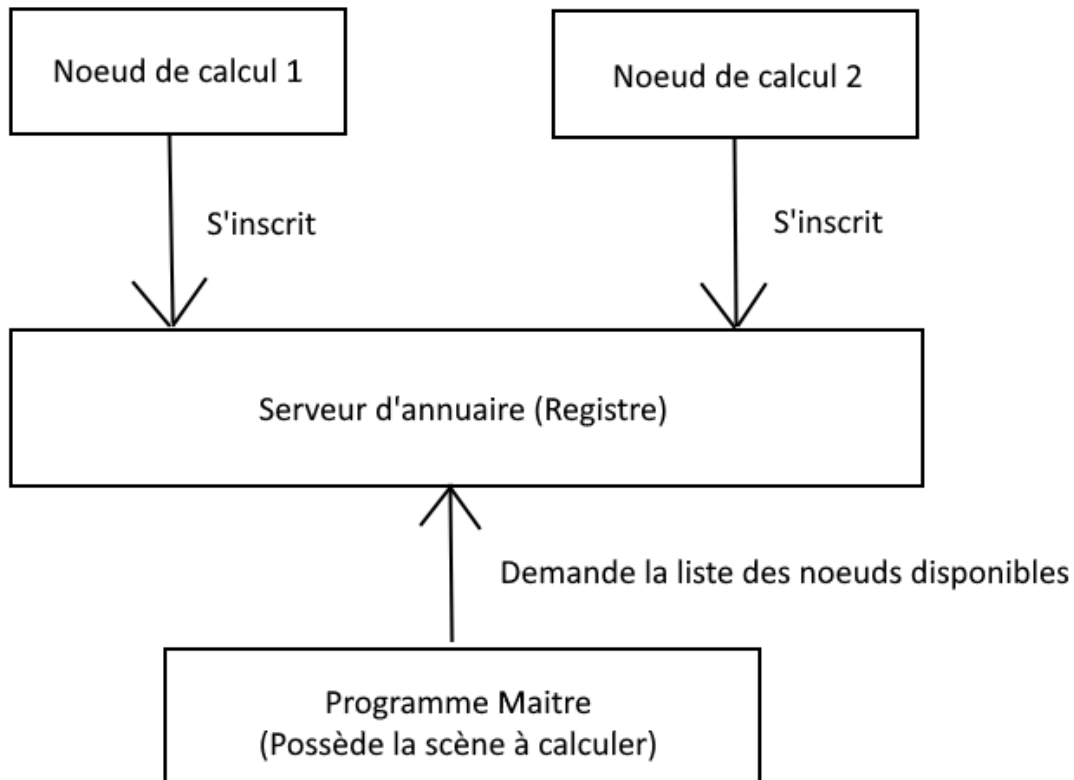
Il renvoie ensuite :

- les pixels calculés
- la position de la zone dans l'image finale

Pour un point de vue plus avancé, voici les trois acteurs dont nous aurons besoins :

1. **L'Annuaire** : C'est le point central de l'application. Il tient simplement à jour la liste des machines.

2. **Les Nœuds de calcul** : Ce sont les ordinateurs dont le seul but est d'attendre qu'on leur donne un bout d'image à calculer.
3. **Le Programme principal** : C'est lui qui possède le fichier de la scène à calculer, qui va découper le travail, le distribuer, et tout assembler à la fin.



Scénario de fonctionnement étape par étape :

- **Étape 1** : Les nœuds de calcul démarrent sur différents terminaux. Ils contactent le Serveur d'Annuaire pour s'inscrire.
- **Étape 2** : Le programme principal démarre. Il contacte l'Annuaire et récupère les adresses de tous les nœuds de calcul disponibles.
- **Étape 3** : Le programme principal divise l'image finale en plusieurs morceaux.
- **Étape 4** : Le programme principal envoie à chaque nœud la description de la scène et la zone spécifique qu'il doit calculer.
- **Étape 5** : Tous les nœuds travaillent en même temps sur leur propre morceau.
- **Étape 6** : Chaque nœud renvoie son bout d'image calculé. Le programme principal fusionne ces sous-images dans une grande image finale et l'affiche à l'écran.

Dans notre cas, chaque nœud de calcul applique le même algorithme de raytracing, mais uniquement sur une partie de l'image. Si les morceaux sont bien répartis et que les machines

travaillent réellement en même temps, le calcul final peut être beaucoup plus rapide qu'avec une seule machine.

Le tracé de rayon (raytracing)

```
PS C:\Users\G614\Documents\git\RMI_Calcul_Paral-lle> java LancerRaytracer simple.txt 100 100
Calcul de l'image :
- Coordonnées : 0,0
- Taille 100x100
Image calculée en :88 ms
```

```
PS C:\Users\G614\Documents\git\RMI_Calcul_Paral-lle> java LancerRaytracer simple.txt 200 200
Calcul de l'image :
- Coordonnées : 0,0
- Taille 200x200
Image calculée en :230 ms
```

```
PS C:\Users\G614\Documents\git\RMI_Calcul_Paral-lle> java LancerRaytracer simple.txt 300 300
Calcul de l'image :
- Coordonnées : 0,0
- Taille 300x300
Image calculée en :453 ms
```

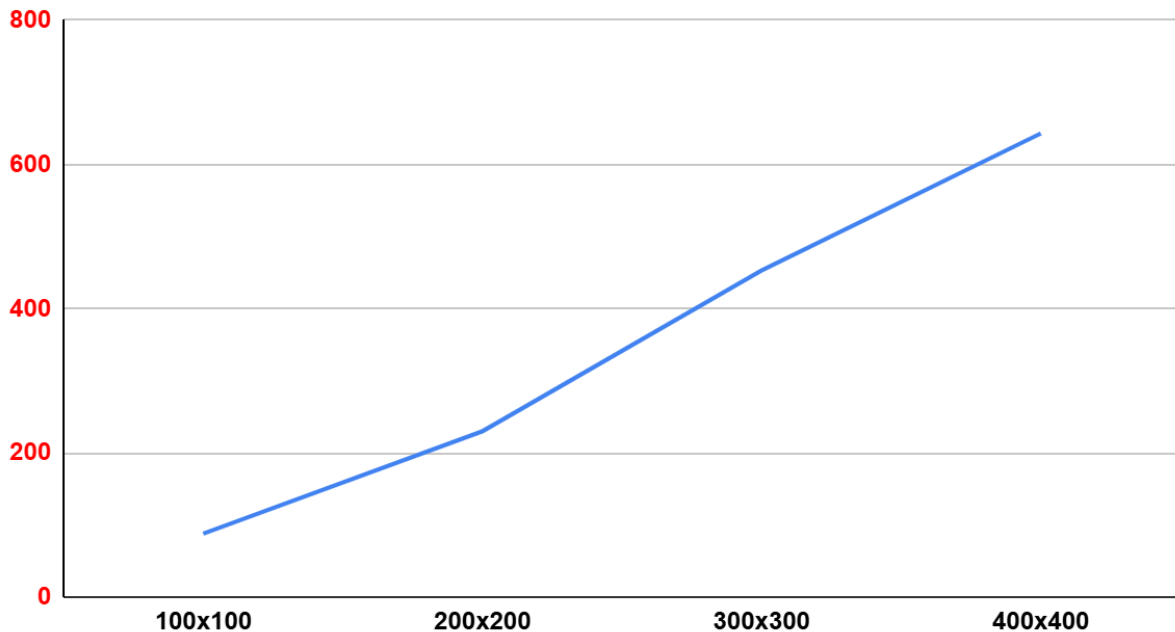
```
PS C:\Users\G614\Documents\git\RMI_Calcul_Paral-lle> java LancerRaytracer simple.txt 400 400
Calcul de l'image :
- Coordonnées : 0,0
- Taille 400x400
Image calculée en :643 ms
```

Cela signifie que pour le même point de coordonné ($x = 0$, $y = 0$) :

Coordonnée	hauteur	largeur	temps de calcul
0,0	100	100	88ms
0,0	200	200	230ms
0,0	300	300	453ms
0,0	400	400	643ms

Donc plus l'image est grande, plus le temps de calcul est long !

Temps d'exécution en millisecondes



Explication :

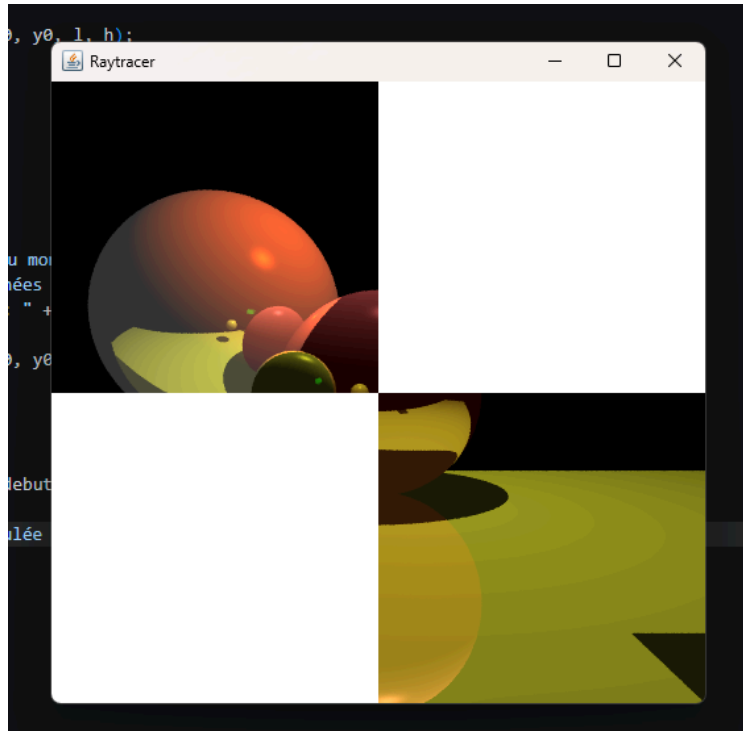
Le raytracing calcule la couleur de l'image pixel par pixel. Si la résolution de l'image double, par exemple, on passe de 500x500 à 1000x1000, le nombre total de pixels ne double pas, il est quadruplé : de 250 000 on passe à 1 000 000 de pixels.

De plus, pour chaque pixel, l'algorithme lance un rayon mathématique et doit calculer les intersections avec toutes les sphères et sources lumineuses de la scène via `simple.txt`.

Ainsi, le temps d'exécution augmente de manière exponentielle ou quadratique par rapport aux dimensions de l'image. C'est exactement ce qui justifie le besoin de paralléliser ce calcul sur plusieurs machines pour les hautes résolutions.

Modification de `LancerRaytracer.java` pour afficher l'image découpée comme demandé.

Pour répondre à cette question sans modifier *RayTracer*, nous avons changé les paramètres de la méthode `scene.compute(x0, y0, l, h)`. Au lieu de calculer l'image globale d'un seul bloc, nous allons déjà tester la capacité à ne calculer que des morceaux de l'image, ce qui est la première étape avant de faire une distribution sur le réseau.



```
Instant debut = Instant.now();

int x0 = 0;
int y0 = 0;
int l = largeur / 2;
int h = hauteur / 2;

System.out.println("Calcul du morceau 1 :");
System.out.println(" - Coordonnées : " + x0 + "," + y0);
System.out.println(" - Taille : " + l + "x" + h);

Image image1 = scene.compute(x0, y0, l, h);
disp.setImage(image1, x0, y0);

x0 = largeur / 2;
y0 = hauteur / 2;
l = largeur / 2;
h = hauteur / 2;

System.out.println("Calcul du morceau 2 :");
System.out.println(" - Coordonnées : " + x0 + "," + y0);
System.out.println(" - Taille : " + l + "x" + h);

Image image2 = scene.compute(x0, y0, l, h);
disp.setImage(image2, x0, y0);

Instant fin = Instant.now();

long duree = Duration.between(debut, fin).toMillis();

System.out.println("Image calculée en : " + duree + " ms");
```

```
Image calculée en : 372 ms
PS C:\Users\G614\Documents\git\RMI_Calcul_Paral-lle> java LancerRaytracer simple.txt 400 400
Calcul du morceau 1 :
- Coordonnées : 0,0
- Taille : 200x200
Calcul du morceau 2 :
- Coordonnées : 200,200
- Taille : 200x200
Image calculée en : 394 ms
█
```

On peut remarquer que générer une partie de l'image est plus rapide que de la générer entièrement !

Explication de la logique algorithmique

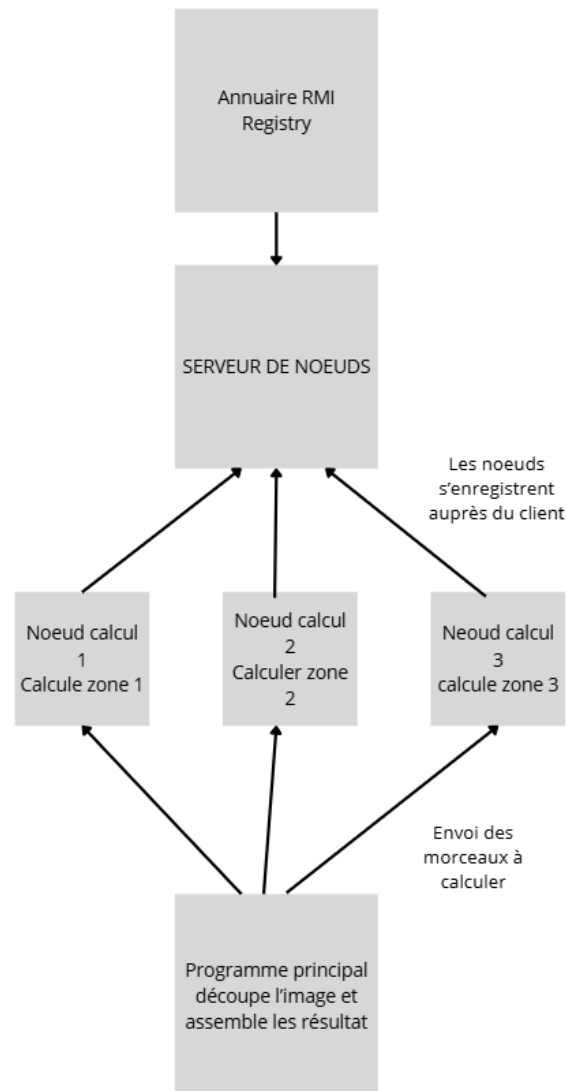
L'algorithme mis en place ici valide la faisabilité du découpage de l'image :

1. On utilise le package *java.time.Instant* pour démarrer un chronomètre juste avant le premier calcul, et l'arrête à la fin pour afficher la durée d'exécution en millisecondes.
2. Pour le découpage :
 - Pour le Morceau 1, l'algorithme définit son point de départ aux coordonnées $x0=0$, $y0=0$ et limite le calcul à une largeur l et une hauteur h divisées par deux.
 - Pour le Morceau 2, l'algorithme déplace le point de départ au centre exact de l'image tout en conservant les mêmes dimensions réduites.
3. La reconstitution s'effectue directement sur l'interface graphique. En utilisant la méthode `disp.setImage(image1, x0, y0)`, chaque bloc calculé indépendamment est replacé à ses coordonnées d'origine sur l'image principale.

Accélérons les choses : L'architecture répartie

A. Schéma de l'architecture et identification des processus

Le sujet demande d'identifier les différents processus impliqués dans notre architecture et leur nature.



Définition processus fixes

Les processus fixes sont ceux qui restent lancés et qui sont connus par les autres programmes.

1. **Le processus fixe** : Le serveur de nœuds est le seul processus fixe de l'architecture. Il écoute sur un port choisi, ici le port 5050, de manière permanente. Il doit être démarré en premier sur une machine connue. C'est lui qui permet de récupérer les coordonnées des nœuds de calcul.

2. **Les processus mobiles** : Les processus mobiles entrent et sortent à leur guise. Il s'agit de :

- **Les nœuds de calcul** : Ce sont des programmes que l'on peut lancer sur n'importe quel ordinateur. Dès qu'ils démarrent, ils s'enregistrent auprès du serveur de nœuds. S'ils sont éteints, ils disparaissent du réseau sans faire planter le serveur.
- **Le programme principal** : C'est le processus utilisateur. Il se connecte au réseau pour récupérer les coordonnées des nœuds disponibles, découpe le calcul, l'envoie sur chacun d'eux, assemble l'image, l'affiche, puis se termine.

B. Les types de données échangées

Pour que la communication fonctionne via RMI, toutes les données échangées entre ces processus doivent obligatoirement implémenter l'interface Serializable.

Émetteur	Récepteur	Donnée envoyée	Rôle
Programme principal	Serveur de nœuds	demande de liste	savoir quels nœuds sont disponibles
Serveur de nœuds	Programme principal	liste de nœuds	permettre la distribution du calcul
Programme principal	Nœud de calcul	scène + zone à calculer	demander le calcul d'un fragment
Programme principal	Nœud de calcul	x0, y0, largeur, hauteur	définir le morceau d'image
Nœud de calcul	Programme principal	Image partielle	résultat du calcul

Programme principal	Affichage	image complète	résultat final assemblé
---------------------	-----------	----------------	-------------------------

Les nœuds de calcul envoient au serveur de nœuds leur référence *ServiceCalcul*. Cette référence permet ensuite au coordinateur de les appeler à distance.

Le coordinateur récupère auprès du serveur une liste de services de calcul disponibles.

Ensuite, pour chaque fragment, le coordinateur envoie à un nœud :

- l'objet Scene ;
- les coordonnées x0 et y0 ;
- la largeur du fragment ;
- la hauteur du fragment.

Le nœud renvoie une image partielle de type Image.

Le coordinateur utilise ensuite `disp.setImage(...)` pour replacer chaque fragment dans l'image finale.

C. Condition requise pour le parallélisme

Question : Si on veut que les calculs se fassent en parallèle, que faut-il faire ?

Si le programme principal utilise une simple boucle *for* pour distribuer le travail aux nœuds, l'exécution RMI sera bloquante. Le programme enverra le morceau 1 au Nœud 1, puis attendra de recevoir le résultat avant de demander au Nœud 2 de calculer le morceau 2. Les nœuds ne travailleront donc pas en même temps, et l'accélération serait nulle.

Pour que les calculs se fassent réellement en parallèle sur l'ensemble des machines, il faut impérativement utiliser le multithreading du côté du programme principal. Le programme principal doit créer un *Thread* indépendant pour chaque nœud de calcul. Ainsi, toutes les requêtes partent sur le réseau exactement au même instant. Tous les nœuds de calcul effectuent leur traitement simultanément, et le programme principal n'a plus qu'à attendre la fin de tous les Threads pour reconstituer l'image finale.

Le principe est le suivant :

```
t1.start();
```

```
t2.start();
```

```
t3.start();
```

```
t1.join();
```

```
t2.join();
```

```
t3.join();
```

Grâce à cela, le temps total de calcul dépend davantage du nœud le plus lent que de la somme des temps de tous les nœuds.

D. Réalisation de l'application

L'application répartie est composée de trois éléments principaux.

- Le premier élément est le serveur de nœuds. Il est lancé par la classe *ServeurNoeuds*. Ce programme crée le registry RMI, crée le service *ServiceNoeudsImpl*, puis l'enregistre dans le registry sous le nom *noeuds*.
- Le deuxième élément est le nœud de calcul. Il est lancé par la classe *NoeudCalcul*. Chaque nœud crée un service *ServiceCalculImpl*, puis s'enregistre auprès du serveur de nœuds avec la méthode *enregistrerNoeud*.
- Le troisième élément est le programme principal, appelé *CoordinateurRaytracer*. Il récupère le service *noeuds* dans le registry, demande la liste des nœuds disponibles avec *getNoeuds*, découpe l'image en fragments, envoie les calculs aux nœuds avec *calculerFragment*, puis assemble les résultats.

Rôle des composants :

- **ServeurNoeuds**
ServeurNoeuds est le processus fixe principal. Il démarre le registry RMI sur le port 1099, crée le service *ServiceNoeudsImpl*, puis l'enregistre dans le registry sous le nom *noeuds*.
C'est le premier programme à lancer.
- **ServiceNoeuds**
ServiceNoeuds est l'interface du serveur de nœuds. Elle propose les méthodes utilisées pour enregistrer les nœuds et récupérer la liste des nœuds disponibles.
- **ServiceNoeudsImpl**
ServiceNoeudsImpl implémente l'interface *ServiceNoeuds*. Cette classe stocke une liste de références vers les services de calcul disponibles.
Quand un nœud se connecte, il appelle : *enregistrerNoeud(serviceCalcul)*
Le serveur ajoute alors ce nœud dans sa liste.
- **NoeudCalcul**
NoeudCalcul est le programme qui lance un nœud de calcul. Il crée une instance de *ServiceCalculImpl*, récupère le service *noeuds* dans le registry, puis enregistre son service de calcul auprès du serveur.
Un nœud peut être lancé plusieurs fois, dans plusieurs terminaux ou sur plusieurs machines.

- **ServiceCalcul**
ServiceCalcul est l'interface proposée par un nœud de calcul. Elle contient la méthode :
`calculerFragment(scene, x0, y0, largeur, hauteur)`
 Cette méthode permet au coordinateur de demander le calcul d'une zone précise de l'image.
- **ServiceCalculImpl**
ServiceCalculImpl est l'implémentation du service de calcul. Elle reçoit une scène et une zone à calculer, puis appelle : `scene.compute(x0, y0, largeur, hauteur)`
 Elle renvoie ensuite l'image partielle calculée.
- **CoordinateurRaytracer**
CoordinateurRaytracer est le programme principal. Il récupère les nœuds disponibles, découpe l'image, lance les calculs en parallèle et assemble les fragments dans la fenêtre finale.

Interfaces utilisées

Interface ServiceCalcul

```
import java.rmi.Remote;

import java.rmi.RemoteException;
```

```
import raytracer.Image;
```

```
import raytracer.Scene;
```

```
public interface ServiceCalcul extends Remote {
```

```
    Image calculerFragment(Scene scene, int x0, int y0, int largeur, int hauteur)
```

```
        throws RemoteException;
```

```
}
```

Cette interface est utilisée par les nœuds de calcul. Elle permet au coordinateur de demander le calcul d'une partie de l'image.

Interface ServiceNoeuds

```
import java.rmi.Remote;
```

```
import java.rmi.RemoteException;
```

```
import java.util.List;
```

```
public interface ServiceNoeuds extends Remote {  
  
    void enregistrerNoeud(ServiceCalcul serviceCalcul) throws RemoteException;  
  
    List<ServiceCalcul> getNoeuds() throws RemoteException;  
}
```

Cette interface est utilisée par le serveur de nœuds. Elle permet aux nœuds de s'enregistrer et au coordinateur de récupérer les nœuds disponibles.

Déroulement complet de l'exécution

L'exécution complète se fait dans cet ordre.

D'abord, on lance *ServeurNoeuds*. Il crée le registry RMI, crée le service *ServiceNoeudsImpl*, puis l'enregistre sous le nom *noeuds*.

Ensuite, on lance un ou plusieurs *NoeudCalcul*. Chaque nœud crée son propre *ServiceCalculImpl*, récupère le service *noeuds* dans le registry, puis s'enregistre auprès du serveur avec *enregistrerNoeud*.

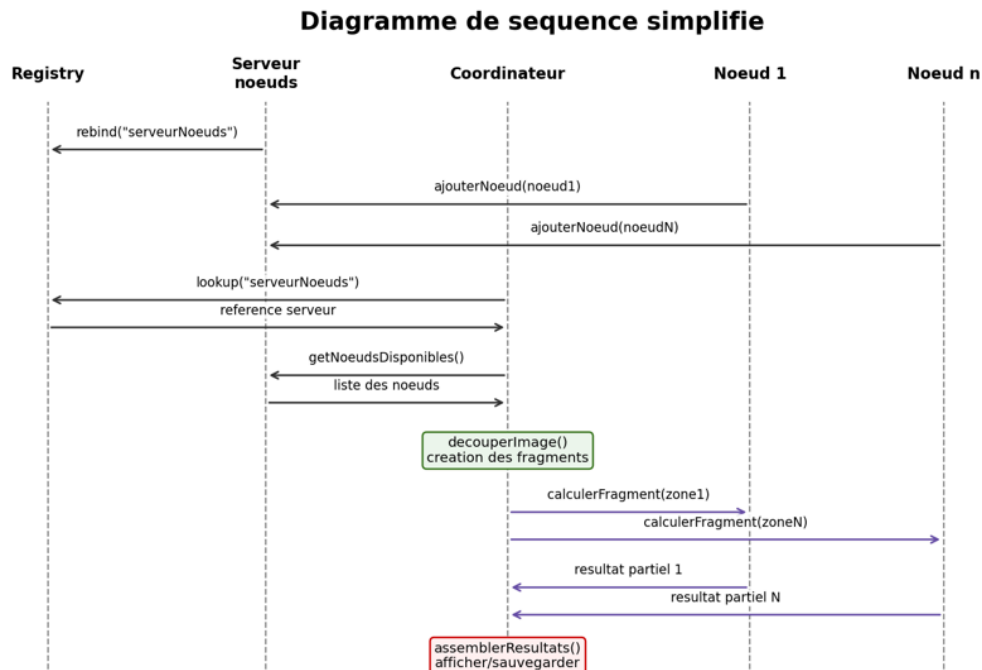
Enfin, on lance *CoordinateurRaytracer*. Il récupère le service *noeuds* dans le registry avec *lookup*, puis appelle *getNoeuds* pour obtenir la liste des services de calcul disponibles.

Le coordinateur découpe ensuite l'image en autant de fragments que de nœuds disponibles. Dans notre implémentation, le découpage se fait en bandes horizontales. Par exemple, avec trois nœuds et une image de 800 pixels de hauteur, les fragments sont répartis sur trois zones.

Chaque fragment est envoyé à un nœud différent avec la méthode *calculerFragment*.

Une fois tous les fragments reçus, le coordinateur les affiche à leur position d'origine pour reconstruire l'image complète.

Diagramme de séquence



Résultats et interprétation

Les tests distribués montrent que les nœuds s'enregistrent correctement auprès du serveur. Le coordinateur récupère bien le nombre de nœuds disponibles et envoie plusieurs fragments.

Avec trois nœuds, le coordinateur affiche plusieurs envois de fragments et reçoit ensuite les résultats. L'ordre d'envoi ou de réception des fragments n'est pas toujours strictement le même, ce qui est normal avec les threads. Cela montre que les calculs sont lancés en parallèle.

Avec plus de nœuds, le coordinateur découpe l'image en plus de fragments. Le temps de calcul peut alors diminuer, mais le gain dépend aussi de la machine utilisée, du coût RMI et du nombre de processus lancés.

Si tous les nœuds sont lancés sur le même ordinateur, l'application montre bien le fonctionnement réparti avec RMI, mais l'accélération peut être limitée car les calculs utilisent quand même les ressources de la même machine.

```
PS C:\Users\G614\Documents\git\RFI_Calcul_Paral-lle> java ServeurNoeuds
Registry créé sur le port 1099
Serveur de n?uds prêt.
Service enregistré sous le nom : noeuds
Nouveau n?ud enregistré.
Nombre de n?uds disponibles : 1
Nouveau n?ud enregistré.
Nombre de n?uds disponibles : 2
Nouveau n?ud enregistré.
Nombre de n?uds disponibles : 3
```

```
PS C:\Users\G614\Documents\git\RFI_Calcul_Paral-lle> java CoordinateurRaytracer simple.txt 800 800
Usage : java CoordinateurRaytracer [fichier-scene] [largeur] [hauteur] [hote] [port]
Nombre de n?uds disponibles : 1
Envoi du fragment 0
- position : 0,0
- taille : 800x800
Fragment 0 reçu.
Image calculée en distribué : 2496 ms
PS C:\Users\G614\Documents\git\RFI_Calcul_Paral-lle> |
```

```
PS C:\Users\G614\Documents\git\RFI_Calcul_Paral-lle> java CoordinateurRaytracer simple.txt 800 800
Usage : java CoordinateurRaytracer [fichier-scene] [largeur] [hauteur] [hote] [port]
Nombre de n?uds disponibles : 3
Envoi du fragment 0
Envoi du fragment 2
Envoi du fragment 1
- position : 0,266
- position : 0,0
- position : 0,532
- taille : 800x266
- taille : 800x266
- taille : 800x268
Fragment 0 reçu.
Fragment 1 reçu.
Fragment 2 reçu.
Image calculée en distribué : 1509 ms
PS C:\Users\G614\Documents\git\RFI_Calcul_Paral-lle> |
```

```

PS C:\Users\G614\Documents\git\RMI_Calcul_Paral-lle> java CoordinateurRaytracer simple.txt 800 800
Usage : java CoordinateurRaytracer [fichier-scene] [largeur] [hauteur] [hote] [port]
Nombre de nœuds disponibles : 6
Envoi du fragment 0
Envoi du fragment 4
- position : 0,532
Envoi du fragment 1
Envoi du fragment 2
- position : 0,266
- taille : 800x133
Envoi du fragment 3
- position : 0,399
- taille : 800x133
- position : 0,0
- taille : 800x133
Envoi du fragment 5
- position : 0,665
- taille : 800x135
- position : 0,133
- taille : 800x133
- taille : 800x133
Fragment 0 reçu.
Fragment 1 reçu.
Fragment 2 reçu.
Fragment 3 reçu.
Fragment 4 reçu.
Fragment 5 reçu.
Image calculée en distribué : 1221 ms
PS C:\Users\G614\Documents\git\RMI_Calcul_Paral-lle> |

```

```

Fragment 1 reçu.
Fragment 2 reçu.
Fragment 0 reçu.
Fragment 5 reçu.
Fragment 3 reçu.
Fragment 4 reçu.
Fragment 7 reçu.
Fragment 10 reçu.
Fragment 8 reçu.
Fragment 11 reçu.
Fragment 9 reçu.
Fragment 6 reçu.
Fragment 13 reçu.
Fragment 12 reçu.
Image calculée en distribué : 2094 ms
PS C:\Users\G614\Documents\git\RMI_Calcul_Paral-lle> |

```

On voit bien que 1 nœud est plus long que 3 ou 6 nœuds. En revanche quand il y a trop de nœuds ça redevient plus long car le temps de recevoir la partie de l'image à calculer car trop nombreux.

Limites de l'application

La solution fonctionne, mais elle possède certaines limites :

- Le serveur de nœuds reste un point central. Si ce serveur n'est pas lancé, les nœuds ne peuvent pas s'enregistrer et le coordinateur ne peut pas récupérer la liste des machines disponibles.
- Si un nœud tombe pendant un calcul, le fragment qui lui a été envoyé peut être perdu. Dans notre version, il n'y a pas encore de redistribution automatique du fragment vers un autre nœud.
- Le réseau ajoute aussi un coût. Pour de petites images, le calcul distribué peut ne pas être beaucoup plus rapide que le calcul local, car le temps gagné peut être compensé par les communications RMI.
- Enfin, le découpage en bandes horizontales est simple, mais il ne tient pas compte de la puissance des machines. Si un nœud est plus lent que les autres, il peut ralentir l'assemblage final.